

My Project

Généré par Doxygen 1.13.2

Chapitre 1

Puissance 4

1.1 Puissance 4

1.1.1 Introduction

Ce projet est un jeu de Puissance 4 en C++.

1.1.2 Classes du projet

Cette documentation contient l'ensemble des classes et structures du projet.

1.1.2.1 Gestion des joueurs

- [Joueur](#)
- [JoueurHumain](#)
- [JoueurIA](#)
- [IA_Aleatoire](#)
- [IA_MinMax](#)
- [JoueurMPI](#)

1.1.2.2 Gestion du jeu

- [Partie](#)
- [Plateau3D](#)
- arbitre
- position

1.1.2.3 Structures et types

- [Vec3](#)
- Pion

1.1.3 Lancement

La documentation des classes est disponible dans l'onglet **Classes**.

1.1.4 Projet

Projet de Puissance 4 pour le TT de Génie Logiciel, MAM 2025-2026

	Poste	Responsable
	Responsable gestion/ répartition écriture , déroulé	ZERZOUH Rym
	Responsable élaboration tests, validation des codes...	HORDOIR Léontine
	Responsable documentation, élaboration de la charte d'écriture	MANCEAU Emilien
	Responsable rapport latex, gestion des commits	LE CORRE Nancy

Chapitre 2

Index hiérarchique

2.1 Hiérarchie des classes

Cette liste d'héritage est classée approximativement par ordre alphabétique :

arbitre	??
Joueur	??
JoueurHumain	??
JoueurIA	??
IA_Aleatoire	??
IA_MinMax	??
JoueurMPI	??
Partie	??
Plateau3D	??
position	??
Vec3	??

Chapitre 3

Index des classes

3.1 Liste des classes

Liste des classes, structures, unions et interfaces avec une brève description :

arbitre	??
IA_Aleatoire	
Intelligence artificielle qui choisit ses coups aléatoirement	??
IA_MinMax	
Intelligence artificielle utilisant l'algorithme MinMax	??
Joueur	
Classe abstraite représentant un joueur	??
JoueurHumain	
Représente un joueur humain	??
JoueurIA	
Représente un joueur contrôlé par une intelligence artificielle	??
JoueurMPI	??
Partie	
Gère le déroulement d'une partie de Puissance 4	??
Plateau3D	
Représente le plateau de jeu du Puissance 4 en 3D	??
position	??
Vec3	
Représente une direction en 3 dimensions	??

Chapitre 4

Index des fichiers

4.1 Liste des fichiers

Liste de tous les fichiers documentés avec une brève description :

src/ arbitre.h	??
src/ IA_Aleatoire.h	??
src/ IA_MinMax.h	??
src/ Joueur.h	??
src/ JoueurHumain.h	??
src/ JoueurIA.h	??
src/ JoueurMPI.h	??
src/ main.cpp	
Programme principal utilisant MPI	??
src/ Partie.h	??
src/ Pion.hpp	??
src/ Plateau3D.h	??
src/ position.h	??
src/ test.cpp	
Fichier de test pour comparer les IA	??
src/ TestJeu.cpp	
Ancien fichier de test pour les communications MPI	??

Chapitre 5

Documentation des classes

5.1 Référence de la classe arbitre

Fonctions membres publiques

- **arbitre** (const [arbitre](#) &other)
- [arbitre](#) & **operator=** (const [arbitre](#) &other)
- [position](#) **receiveCoup** (int rank)
- void **sendCoup** ([position](#), int rank)

La documentation de cette classe a été générée à partir des fichiers suivants :

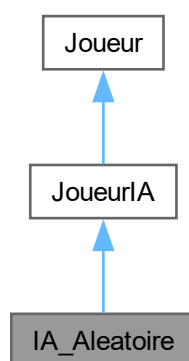
- src/arbitre.h
- src/arbitre.cpp

5.2 Référence de la classe IA_Aleatoire

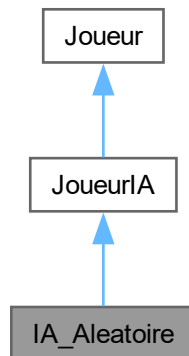
Intelligence artificielle qui choisit ses coups aléatoirement.

```
#include <IA_Aleatoire.h>
```

Graphe d'héritage de IA_Aleatoire:



Graphe de collaboration de IA_Aleatoire:



Fonctions membres publiques

- [IA_Aleatoire](#) (Pion [couleur](#))
Constructeur avec couleur.
- [IA_Aleatoire](#) (string [nom](#), Pion [couleur](#))
Constructeur avec nom et couleur.
- [IA_Aleatoire](#) (const [IA_Aleatoire](#) &other)
Constructeur par copie.
- [~IA_Aleatoire](#) ()
Destructeur.
- [IA_Aleatoire](#) & [operator=](#) (const [IA_Aleatoire](#) &other)
Opérateur d'affectation.
- std::pair< int, int > [choisirCoup](#) (const [Plateau3D](#) &plateau)
Choisit un coup valide au hasard.

Fonctions membres publiques hérités de [JoueurIA](#)

- [JoueurIA](#) (Pion [couleur](#))
Constructeur avec couleur.
- [JoueurIA](#) (string [nom](#), Pion [couleur](#))
Constructeur avec nom et couleur.
- [JoueurIA](#) (const [JoueurIA](#) &other)
Constructeur par copie.
- [~JoueurIA](#) ()
Destructeur.
- [JoueurIA](#) & [operator=](#) (const [JoueurIA](#) &other)
Opérateur d'affectation.

Fonctions membres publiques hérités de [Joueur](#)

- [Joueur](#) (std::string [nom](#), Pion [couleur](#))
- [Joueur](#) (Pion [couleur](#))
- [Joueur](#) (const [Joueur](#) &other)
- [Joueur](#) & [operator=](#) (const [Joueur](#) &other)
- std::string [getNom](#) () const
Retourne le nom du joueur.
- Pion [getCouleur](#) () const
Retourne la couleur du joueur.

Attributs protégés

- mt19937 **moteur**
Générateur de nombres aléatoires.

Attributs protégés hérités de [Joueur](#)

- std::string **nom**
Nom du joueur.
- Pion **couleur**
Couleur du pion du joueur.

5.2.1 Description détaillée

Intelligence artificielle qui choisit ses coups aléatoirement.

Cette classe parcourt les coups possibles sur le plateau, puis en sélectionne un au hasard.

5.2.2 Documentation des constructeurs et destructeur**5.2.2.1 IA_Aleatoire() [1/3]**

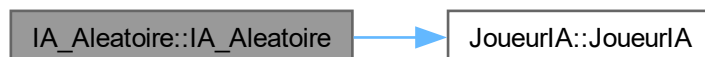
```
IA_Aleatoire::IA_Aleatoire (
    Pion couleur)
```

Constructeur avec couleur.

Paramètres

<code>couleur</code>	Couleur du pion de l'IA.
----------------------	--------------------------

Voici le graphe d'appel pour cette fonction :

**5.2.2.2 IA_Aleatoire() [2/3]**

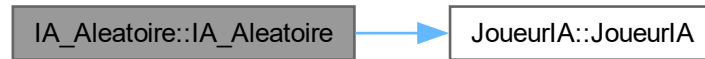
```
IA_Aleatoire::IA_Aleatoire (
    string nom,
    Pion couleur)
```

Constructeur avec nom et couleur.

Paramètres

<code>nom</code>	Nom de l'IA.
<code>couleur</code>	Couleur du pion de l'IA.

Voici le graphe d'appel pour cette fonction :



5.2.2.3 IA_Aleatoire() [3/3]

```
IA_Aleatoire::IA_Aleatoire (
    const IA_Aleatoire & other)
```

Constructeur par copie.

Paramètres

<code>other</code>	<code>IA_Aleatoire</code> à copier.
--------------------	-------------------------------------

Voici le graphe d'appel pour cette fonction :



5.2.3 Documentation des fonctions membres

5.2.3.1 choisirCoup()

```
std::pair< int, int > IA_Aleatoire::choisirCoup (
    const Plateau3D & plateau) [virtual]
```

Choisit un coup valide au hasard.

Paramètres

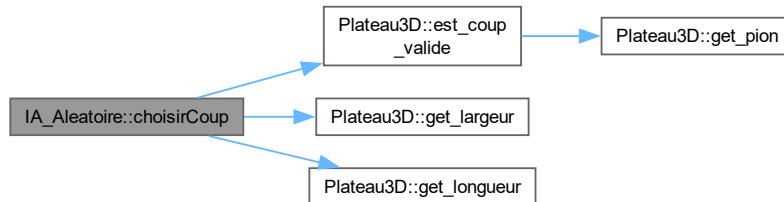
<code>plateau</code>	Plateau de jeu.
----------------------	-----------------

Renvoie

Paire d'entiers représentant le coup choisi.

Implémente [Joueur](#).

Voici le graphe d'appel pour cette fonction :

**5.2.3.2 operator=()**

```
IA_Aleatoire & IA_Aleatoire::operator= (
    const IA_Aleatoire & other)
```

Opérateur d'affectation.

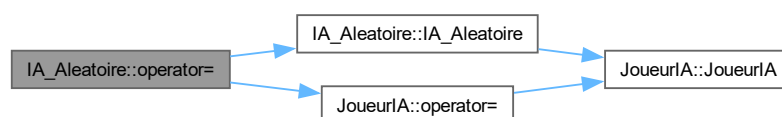
Paramètres

<code>other</code>	IA_Aleatoire à copier.
--------------------	--

Renvoie

Référence vers l'IA courante.

Voici le graphe d'appel pour cette fonction :



La documentation de cette classe a été générée à partir des fichiers suivants :

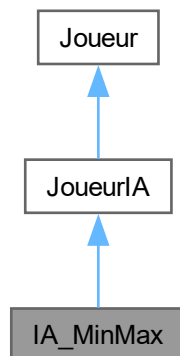
- `src/IA_Aleatoire.h`
- `src/IA_Aleatoire.cpp`

5.3 Référence de la classe IA_MinMax

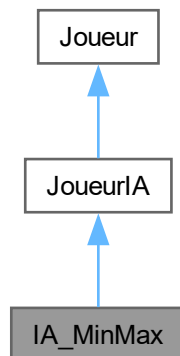
Intelligence artificielle utilisant l'algorithme MinMax.

```
#include <IA_MinMax.h>
```

Graphe d'héritage de IA_MinMax:



Graphe de collaboration de IA_MinMax:



Fonctions membres publiques

- `IA_MinMax` (std::string `nom`, Pion `couleur`, int ProfondeurEntree)
Constructeur avec nom, couleur et profondeur.
- `IA_MinMax` (Pion `couleur`)
Constructeur avec couleur.
- `IA_MinMax` (const `IA_MinMax` &other)
Constructeur par copie.
- `~IA_MinMax` ()
Destructeur.
- `IA_MinMax` & `operator=` (const `IA_MinMax` &other)
Opérateur d'affectation.
- std::pair< int, int > `choisirCoup` (const `Plateau3D` &plateau)
Choisit le meilleur coup selon l'algorithme MinMax.
- int `evaluerPlateau` (const `Plateau3D` &plateau)

- *Évalue l'état global du plateau.*
- int **evaluer** (int nbrPions, int Pions_adverse) const
Évalue une ligne de quatre cases.
- int **min_max** (Plateau3D &plateau, int **profondeur**, int alpha, int beta, bool estMax)
Applique l'algorithme MinMax avec élagage alpha-bêta.

Fonctions membres publiques hérités de **JoueurIA**

- **JoueurIA** (Pion **couleur**)
Constructeur avec couleur.
- **JoueurIA** (string **nom**, Pion **couleur**)
Constructeur avec nom et couleur.
- **JoueurIA** (const **JoueurIA** &other)
Constructeur par copie.
- **~JoueurIA** ()
Destructeur.
- **JoueurIA** & **operator=** (const **JoueurIA** &other)
Opérateur d'affectation.

Fonctions membres publiques hérités de **Joueur**

- **Joueur** (std::string **nom**, Pion **couleur**)
- **Joueur** (Pion **couleur**)
- **Joueur** (const **Joueur** &other)
- **Joueur** & **operator=** (const **Joueur** &other)
- std::string **getNom** () const
Retourne le nom du joueur.
- Pion **getCouleur** () const
Retourne la couleur du joueur.

Attributs protégés

- int **profondeur**
Profondeur de recherche de l'algorithme.

Attributs protégés hérités de **Joueur**

- std::string **nom**
Nom du joueur.
- Pion **couleur**
Couleur du pion du joueur.

Attributs protégés statiques

- static constexpr int **ordreX** [] = {2, 1, 3, 0, 4}
- static constexpr int **ordreY** [] = {1, 2, 0, 3}

5.3.1 Description détaillée

Intelligence artificielle utilisant l'algorithme MinMax.

Cette IA évalue les coups possibles sur le plateau afin de choisir le coup ayant le meilleur score. Elle utilise aussi l'élagage alpha-bêta pour limiter le nombre de situations à explorer.

5.3.2 Documentation des constructeurs et destructeur

5.3.2.1 IA_MinMax() [1/3]

```
IA_MinMax::IA_MinMax (
    std::string nom,
```

```
Pion couleur,  
int ProfondeurEntree)
```

Constructeur avec nom, couleur et profondeur.

Paramètres

	<i>nom</i>	Nom de l'IA.
	<i>couleur</i>	Couleur du pion de l'IA.
	<i>ProfondeurEntree</i>	Profondeur de recherche de l'algorithme.

Voici le graphe d'appel pour cette fonction :



5.3.2.2 IA_MinMax() [2/3]

```
IA_MinMax::IA_MinMax (
    Pion couleur)
```

Constructeur avec couleur.

Paramètres

	<i>couleur</i>	Couleur du pion de l'IA.
--	----------------	--------------------------

Voici le graphe d'appel pour cette fonction :



5.3.2.3 IA_MinMax() [3/3]

```
IA_MinMax::IA_MinMax (
    const IA_MinMax & other)
```

Constructeur par copie.

Paramètres

	<i>other</i>	IA_MinMax à copier.
--	--------------	---------------------

Voici le graphe d'appel pour cette fonction :



5.3.3 Documentation des fonctions membres

5.3.3.1 choisirCoup()

```
std::pair< int, int > IA_MinMax::choisirCoup (
    const Plateau3D & plateau) [virtual]
```

Choisit le meilleur coup selon l'algorithme MinMax.

Paramètres

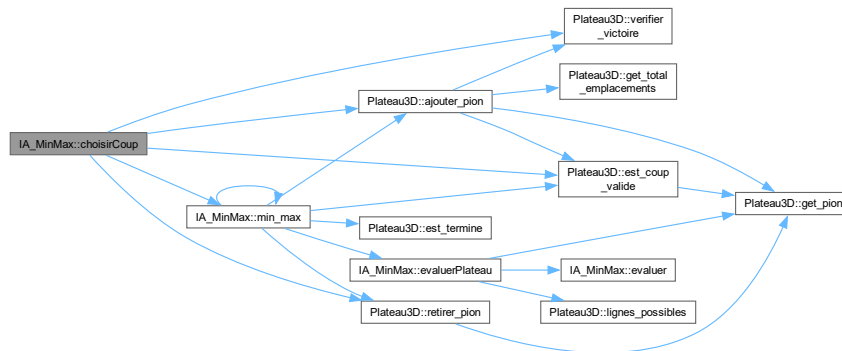
<code>plateau</code>	Plateau de jeu.
----------------------	-----------------

Renvoie

Paire d'entiers représentant le coup choisi.

Implémente [Joueur](#).

Voici le graphe d'appel pour cette fonction :



5.3.3.2 evaluer()

```
int IA_MinMax::evaluer (
    int nbrPions,
    int Pions_adverse) const
```

Évalue une ligne de quatre cases.

Paramètres

<code>nbrPions</code>	Nombre de pions de l'IA.
<code>Pions_adverse</code>	Nombre de pions adverses.

Renvoie

Score de la ligne.

5.3.3.3 evaluerPlateau()

```
int IA_MinMax::evaluerPlateau (
    const Plateau3D & plateau)
```

Évalue l'état global du plateau.

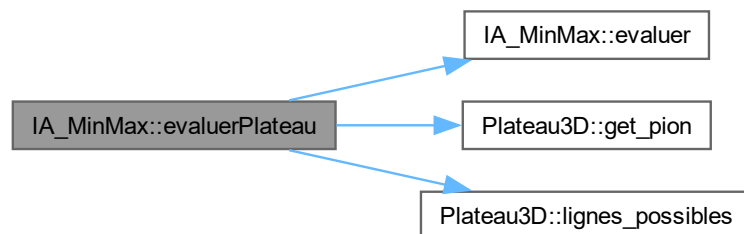
Paramètres

<code>plateau</code>	Plateau à évaluer.
----------------------	--------------------

Renvoie

Score associé au plateau.

Voici le graphe d'appel pour cette fonction :

**5.3.3.4 min_max()**

```
int IA_MinMax::min_max (
    Plateau3D & plateau,
    int profondeur,
    int alpha,
    int beta,
    bool estMax)
```

Applique l'algorithme MinMax avec élagage alpha-bêta.

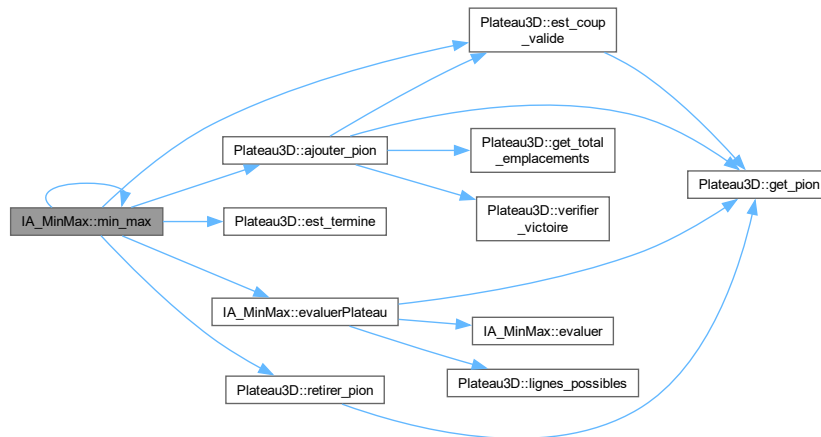
Paramètres

<code>plateau</code>	Plateau simulé.
<code>profondeur</code>	Profondeur restante.
<code>alpha</code>	Meilleur score déjà trouvé pour le joueur maximisant.
<code>beta</code>	Meilleur score déjà trouvé pour le joueur minimisant.
<code>estMax</code>	Indique si le tour simulé est celui de l'IA.

Renvoie

Score du meilleur coup trouvé.

Voici le graphe d'appel pour cette fonction :

**5.3.3.5 operator=()**

```
IA_MinMax & IA_MinMax::operator= (
    const IA_MinMax & other)
```

Opérateur d'affectation.

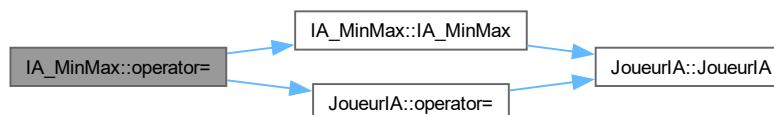
Paramètres

<i>other</i>	IA_MinMax à copier.
--------------	-------------------------------------

Renvoie

Référence vers l'IA courante.

Voici le graphe d'appel pour cette fonction :



La documentation de cette classe a été générée à partir des fichiers suivants :

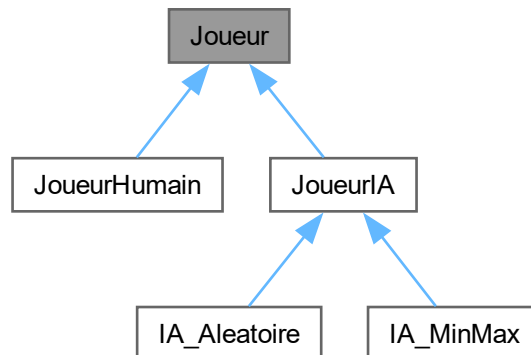
- `src/IA_MinMax.h`
- `src/IA_MinMax.cpp`

5.4 Référence de la classe Joueur

Classe abstraite représentant un joueur.

```
#include <Joueur.h>
```

Graphe d'héritage de Joueur:



Fonctions membres publiques

- **Joueur** (std::string *nom*, Pion *couleur*)
- **Joueur** (Pion *couleur*)
- **Joueur** (const **Joueur** &other)
- **Joueur & operator=** (const **Joueur** &other)
- virtual std::pair< int, int > **choisirCoup** (const **Plateau3D** &plateau)=0
Demande au joueur de choisir un coup.
- std::string **getNom** () const
Retourne le nom du joueur.
- Pion **getCouleur** () const
Retourne la couleur du joueur.

Attributs protégés

- std::string **nom**
Nom du joueur.
- Pion **couleur**
Couleur du pion du joueur.

5.4.1 Description détaillée

Classe abstraite représentant un joueur.

Cette classe contient les informations communes à tous les joueurs, comme le nom et la couleur du pion. Elle impose aussi la méthode **choisirCoup()**, qui sera redéfinie par les classes filles.

5.4.2 Documentation des fonctions membres

5.4.2.1 choisirCoup()

```
virtual std::pair< int, int > Joueur::choisirCoup (
    const Plateau3D & plateau) [pure virtual]
```

Demande au joueur de choisir un coup.

Paramètres

<i>plateau</i>	Plateau de jeu.
----------------	-----------------

Renvoie

Paire d'entiers représentant le coup choisi.

Implémenté dans [IA_Aleatoire](#), [IA_MinMax](#), et [JoueurHumain](#).

5.4.2.2 getCouleur()

```
Pion Joueur::getCouleur () const
```

Retourne la couleur du joueur.

Renvoie

Couleur du pion du joueur.

5.4.2.3 getNom()

```
std::string Joueur::getNom () const
```

Retourne le nom du joueur.

Renvoie

Nom du joueur.

La documentation de cette classe a été générée à partir des fichiers suivants :

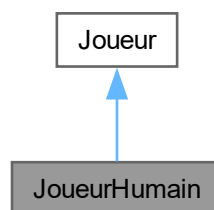
- [src/Joueur.h](#)
- [src/Joueur.cpp](#)

5.5 Référence de la classe JoueurHumain

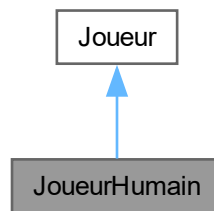
Représente un joueur humain.

```
#include <JoueurHumain.h>
```

Graphe d'héritage de JoueurHumain:



Graphe de collaboration de JoueurHumain:



Fonctions membres publiques

- `JoueurHumain` (string `nom`, Pion `couleur`)
Constructeur avec nom et couleur.
- `JoueurHumain` (const `JoueurHumain` &other)
Constructeur par copie.
- `~JoueurHumain` ()
Destructeur.
- `JoueurHumain` & `operator=` (const `JoueurHumain` &other)
Opérateur d'affectation.
- `std::pair< int, int > choisirCoup` (const `Plateau3D` &plateau)
Demande au joueur humain de choisir un coup.

Fonctions membres publiques hérités de `Joueur`

- `Joueur` (std::string `nom`, Pion `couleur`)
- `Joueur` (Pion `couleur`)
- `Joueur` (const `Joueur` &other)
- `Joueur` & `operator=` (const `Joueur` &other)
- `std::string getNom` () const
Retourne le nom du joueur.
- Pion `getCouleur` () const
Retourne la couleur du joueur.

Membres hérités additionnels

Attributs protégés hérités de `Joueur`

- `std::string nom`
Nom du joueur.
- Pion `couleur`
Couleur du pion du joueur.

5.5.1 Description détaillée

Représente un joueur humain.

Cette classe permet à un utilisateur de choisir ses coups au clavier.

5.5.2 Documentation des constructeurs et destructeur

5.5.2.1 JoueurHumain() [1/2]

```
JoueurHumain::JoueurHumain (
    string nom,
    Pion couleur)
```

Constructeur avec nom et couleur.

Paramètres

<code>nom</code>	Nom du joueur.
<code>couleur</code>	Couleur du pion du joueur.

5.5.2.2 JoueurHumain() [2/2]

```
JoueurHumain::JoueurHumain (
    const JoueurHumain & other)
```

Constructeur par copie.

Paramètres

<code>other</code>	JoueurHumain à copier.
--------------------	--

Voici le graphe d'appel pour cette fonction :



5.5.3 Documentation des fonctions membres

5.5.3.1 choisirCoup()

```
std::pair< int, int > JoueurHumain::choisirCoup (
    const Plateau3D & plateau) [virtual]
```

Demande au joueur humain de choisir un coup.

Paramètres

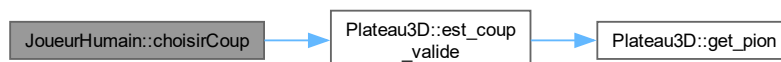
<code>plateau</code>	Plateau de jeu.
----------------------	-----------------

Renvoie

Paire d'entiers représentant le coup choisi.

Implémente [Joueur](#).

Voici le graphe d'appel pour cette fonction :



5.5.3.2 operator=()

```
JoueurHumain & JoueurHumain::operator= (
    const JoueurHumain & other)
```

Opérateur d'affectation.

Paramètres

<i>other</i>	JoueurHumain à copier.
--------------	------------------------

Renvoie

Référence vers le joueur humain courant.

Voici le graphe d'appel pour cette fonction :



La documentation de cette classe a été générée à partir des fichiers suivants :

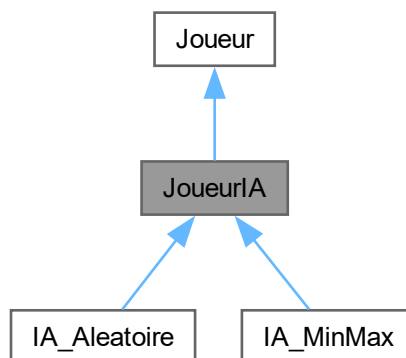
- `src/JoueurHumain.h`
- `src/JoueurHumain.cpp`

5.6 Référence de la classe JoueurIA

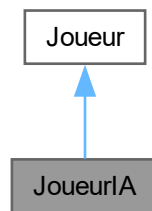
Représente un joueur contrôlé par une intelligence artificielle.

```
#include <JoueurIA.h>
```

Grappe d'héritage de JoueurIA:



Graphe de collaboration de JoueurIA:



Fonctions membres publiques

- **JoueurIA** (Pion **couleur**)
Constructeur avec couleur.
- **JoueurIA** (string **nom**, Pion **couleur**)
Constructeur avec nom et couleur.
- **JoueurIA** (const **JoueurIA** &other)
Constructeur par copie.
- **~JoueurIA** ()
Destructeur.
- **JoueurIA** & **operator=** (const **JoueurIA** &other)
Opérateur d'affectation.

Fonctions membres publiques hérités de **Joueur**

- **Joueur** (std::string **nom**, Pion **couleur**)
- **Joueur** (Pion **couleur**)
- **Joueur** (const **Joueur** &other)
- **Joueur** & **operator=** (const **Joueur** &other)
- virtual std::pair< int, int > **choisirCoup** (const **Plateau3D** &plateau)=0
Demande au joueur de choisir un coup.
- std::string **getNom** () const
Retourne le nom du joueur.
- Pion **getCouleur** () const
Retourne la couleur du joueur.

Membres hérités additionnels

Attributs protégés hérités de **Joueur**

- std::string **nom**
Nom du joueur.
- Pion **couleur**
Couleur du pion du joueur.

5.6.1 Description détaillée

Représente un joueur contrôlé par une intelligence artificielle.

Cette classe sert de base aux différentes IA du projet, comme l'IA aléatoire ou l'IA MinMax.

5.6.2 Documentation des constructeurs et destructeur

5.6.2.1 JoueurIA() [1/3]

```
JoueurIA::JoueurIA (
    Pion couleur)
```

Constructeur avec couleur.

Paramètres

	<i>couleur</i>	Couleur du pion de l'IA.
--	----------------	--------------------------

5.6.2.2 JoueurIA() [2/3]

```
JoueurIA::JoueurIA (
    string nom,
    Pion couleur)
```

Constructeur avec nom et couleur.

Paramètres

	<i>nom</i>	Nom de l'IA.
	<i>couleur</i>	Couleur du pion de l'IA.

5.6.2.3 JoueurIA() [3/3]

```
JoueurIA::JoueurIA (
    const JoueurIA & other)
```

Constructeur par copie.

Paramètres

	<i>other</i>	JoueurIA à copier.
--	--------------	--------------------

Voici le graphe d'appel pour cette fonction :



5.6.3 Documentation des fonctions membres

5.6.3.1 operator=()

```
JoueurIA & JoueurIA::operator= (
    const JoueurIA & other)
```

Opérateur d'affectation.

Paramètres

	<i>other</i>	JoueurIA à copier.
--	--------------	--------------------

Renvoi

Référence vers l'IA courante.

Voici le graphe d'appel pour cette fonction :



La documentation de cette classe a été générée à partir des fichiers suivants :

- src/JoueurIA.h
- src/JoueurIA.cpp

5.7 Référence de la classe JoueurMPI

Fonctions membres publiques

- **JoueurMPI** (int color)
color of the player 0 or 1
- **JoueurMPI** (const joueur &other)
- **JoueurMPI & operator=** (const joueur &other)
- **position receiveCoup** (int=2)
- void **sendCoup** (**position**, int=2)

La documentation de cette classe a été générée à partir des fichiers suivants :

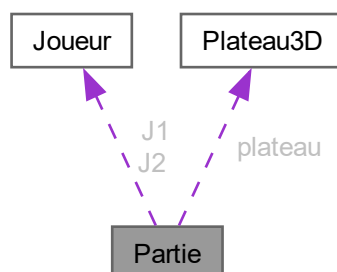
- src/JoueurMPI.h
- src/JoueurMPI.cpp

5.8 Référence de la classe Partie

Gère le déroulement d'une partie de Puissance 4.

```
#include <Partie.h>
```

Grphe de collaboration de Partie:



Fonctions membres publiques

- **Partie** ()
Constructeur par défaut.
- **Partie** (Joueur *J1, Joueur *J2)
Constructeur avec deux joueurs.
- **Partie** (const **Partie** &other)
Constructeur par copie.
- **~Partie** ()
Destructeur.
- **Partie** & **operator=** (const **Partie** &other)
Opérateur égal.
- bool **victoire_tour** (Joueur *j)
Joue le tour d'un joueur et vérifie s'il gagne.
- void **lancer** ()
Lance le déroulement complet de la partie.
- Joueur * **lancer_silencieux** (bool j1commence)
Lance une partie sans affichage détaillé.
- const **Plateau3D** & **getPlateau** () const
Retourne le plateau de la partie.

Attributs protégés

- Joueur * **J1**
Premier joueur.
- Joueur * **J2**
Deuxième joueur.
- **Plateau3D** **plateau**
Plateau de jeu.
- int **tours_joues** = 0
Nombre de tours joués.

5.8.1 Description détaillée

Gère le déroulement d'une partie de Puissance 4.

La classe **Partie** est responsable de :

- la gestion des joueurs
- le suivi du plateau de jeu
- l'alternance des tours
- la détection de fin de partie

5.8.2 Documentation des constructeurs et destructeur

5.8.2.1 **Partie**() [1/2]

```
Partie::Partie (
    Joueur * J1,
    Joueur * J2)
```

Constructeur avec deux joueurs.

Paramètres

J1	Premier joueur.
J2	Deuxième joueur.

5.8.2.2 Partie() [2/2]

```
Partie::Partie (  
    const Partie & other)
```

Constructeur par copie.

Paramètres

<code>other</code>	<code>Partie</code> à copier.
--------------------	-------------------------------

Voici le graphe d'appel pour cette fonction :



5.8.3 Documentation des fonctions membres

5.8.3.1 getPlateau()

```
const Plateau3D & Partie::getPlateau () const [inline]
```

Retourne le plateau de la partie.

Renvoie

Référence constante vers le plateau.

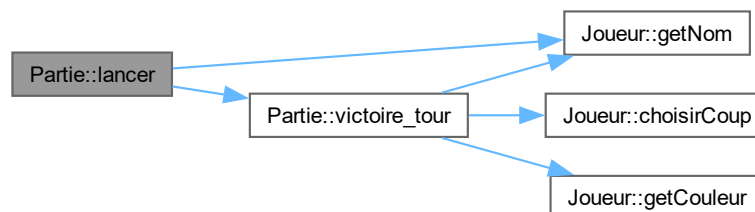
5.8.3.2 lancer()

```
void Partie::lancer ()
```

Lance le déroulement complet de la partie.

Cette méthode alterne les tours entre les deux joueurs jusqu'à ce qu'un joueur gagne ou que la partie se termine.

Voici le graphe d'appel pour cette fonction :



5.8.3.3 lancer_silencieux()

```
Joueur * Partie::lancer_silencieux (
    bool j1commence)
```

Lance une partie sans affichage détaillé.

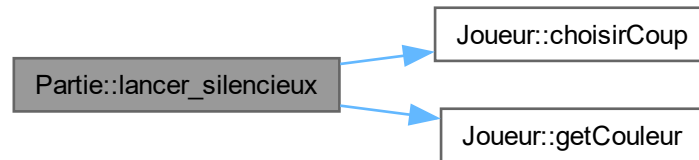
Paramètres

<code>j1commence</code>	Indique si le joueur 1 commence.
-------------------------	----------------------------------

Renvoie

Pointeur vers le joueur gagnant, ou nullptr en cas de match nul.

Voici le graphe d'appel pour cette fonction :

**5.8.3.4 operator=()**

```
Partie & Partie::operator= (  
    const Partie & other)
```

Opérateur égal.

Paramètres

<i>other</i>	<code>Partie</code> à copier.
--------------	-------------------------------

Renvoie

Référence vers la partie courante.

Voici le graphe d'appel pour cette fonction :

**5.8.3.5 victoire_tour()**

```
bool Partie::victoire_tour (  
    Joueur * j)
```

Joue le tour d'un joueur et vérifie s'il gagne.

Le joueur choisit un coup, celui-ci est joué sur le plateau, puis la fonction vérifie si ce coup provoque une victoire.

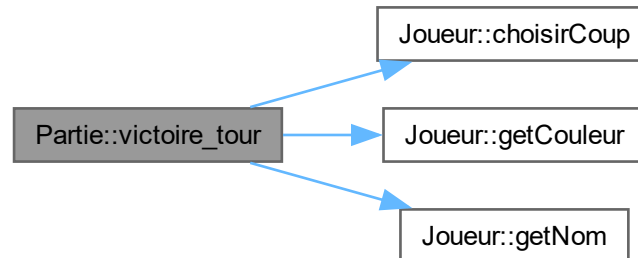
Paramètres

<i>j</i>	Pointeur vers le joueur qui joue le tour.
----------	---

Renvoie

true si le joueur gagne après son coup, false sinon.

Voici le graphe d'appel pour cette fonction :



La documentation de cette classe a été générée à partir des fichiers suivants :

- src/Partie.h
- src/Partie.cpp

5.9 Référence de la classe Plateau3D

Représente le plateau de jeu du Puissance 4 en 3D.

```
#include <Plateau3D.h>
```

Fonctions membres publiques

- [Plateau3D](#) ()
Constructeur par défaut.
- [~Plateau3D](#) ()
Destructeur.
- [Plateau3D](#) (const [Plateau3D](#) &other)
*Constructeur par copie. *.*
- [Plateau3D](#) & [operator=](#) (const [Plateau3D](#) &other)
*Opérateur d'affectation. *.*
- bool [est_coup_valide](#) (int x, int y) const
*Vérifie si un coup est valide. *.*
- int [ajouter_pion](#) (int longueur, int largeur, Pion p)
Ajoute un pion sur le plateau.
- int [retirer_pion](#) (int longueur, int largeur)
*Retire le dernier pion d'une colonne. *.*
- void [affiche](#) ()
Affiche le plateau dans la console.
- bool [verifier_victoire](#) (int x, int y, int z, Pion p) const
*Vérifie si un joueur a gagné. *.*
- int [get_largeur](#) () const
- int [get_longueur](#) () const
- int [get_total_emplacements](#) () const
- std::vector< std::vector< int > > [lignes_possibles](#) () const
*Retourne l'ensemble des lignes gagnantes possibles. *.*
- Pion [get_pion](#) (int hauteur, int largeur, int longueur) const
*Accède à un pion du plateau. *.*
- Pion [get_pion](#) (int index) const

- *Accède à un pion via son indice. *.*
- bool `est_terminé()`
*Indique si la partie est terminée. *.*

5.9.1 Description détaillée

Représente le plateau de jeu du Puissance 4 en 3D.

Cette classe gère la grille, l'ajout et le retrait des pions, ainsi que la vérification des coups et des victoires.

5.9.2 Documentation des constructeurs et destructeur

5.9.2.1 Plateau3D() [1/2]

```
Plateau3D::Plateau3D ()
```

Constructeur par défaut.

•

5.9.2.2 ~Plateau3D()

```
Plateau3D::~~Plateau3D ()
```

Destructeur.

•

5.9.2.3 Plateau3D() [2/2]

```
Plateau3D::Plateau3D (
    const Plateau3D & other)
```

Constructeur par copie. *.

*

Paramètres

<code>other</code>	Plateau à copier.
--------------------	-------------------

Voici le graphe d'appel pour cette fonction :



5.9.3 Documentation des fonctions membres

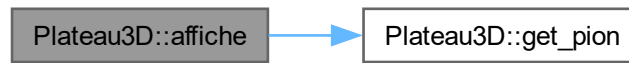
5.9.3.1 affiche()

```
void Plateau3D::affiche ()
```

Affiche le plateau dans la console.

•

Voici le graphe d'appel pour cette fonction :



5.9.3.2 ajouter_pion()

```
int Plateau3D::ajouter_pion (
    int longueur,
    int largeur,
    Pion p)
```

Ajoute un pion sur le plateau.

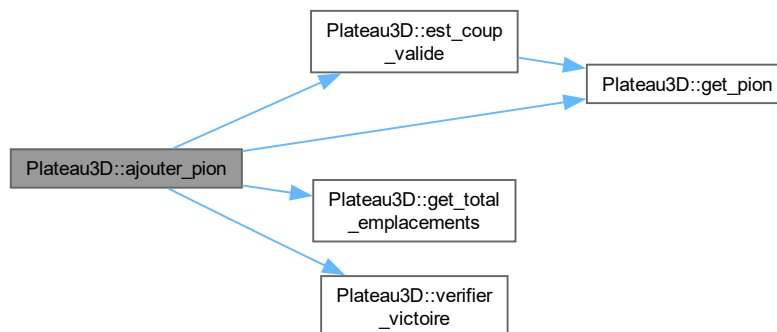
Paramètres

<i>longueur</i>	Coordonnée en longueur.
<i>largeur</i>	Coordonnée en largeur.
<i>p</i>	Pion à ajouter.

Renvoie

Hauteur du pion placé, ou -1 si le coup est invalide.

Voici le graphe d'appel pour cette fonction :



5.9.3.3 est_coup_valide()

```
bool Plateau3D::est_coup_valide (
    int x,
    int y) const
```

Vérifie si un coup est valide. *.

*

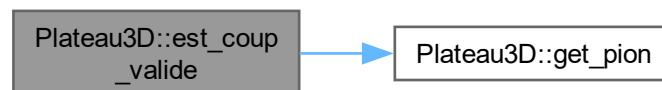
Paramètres

<i>x</i>	Coordonnée en longueur. *
<i>y</i>	Coordonnée en largeur. *

Renvoie

true si le coup est autorisé, false sinon.

Voici le graphe d'appel pour cette fonction :

**5.9.3.4 est_termine()**

```
bool Plateau3D::est_termine ()
```

Indique si la partie est terminée. *.

Renvoie

- true si le plateau est dans un état final.

5.9.3.5 get_largeur()

```
int Plateau3D::get_largeur () const
```

Renvoie

Largeur du plateau.

5.9.3.6 get_longueur()

```
int Plateau3D::get_longueur () const
```

Renvoie

Longueur du plateau.

5.9.3.7 get_pion() [1/2]

```
Pion Plateau3D::get_pion (
    int hauteur,
    int largeur,
    int longueur) const
```

Accède à un pion du plateau. *.

Renvoie

- Pion présent à la position indiquée.

5.9.3.8 get_pion() [2/2]

```
Pion Plateau3D::get_pion (  
    int index) const
```

Accède à un pion via son indice. *.

*

Paramètres

<code>index</code>	Indice dans la grille. *
--------------------	--------------------------

Renvoie

Pion correspondant.

5.9.3.9 get_total_emplacements()

```
int Plateau3D::get_total_emplacements () const
```

Renvoie

Nombre total de cases du plateau.

5.9.3.10 lignes_possibles()

```
std::vector< std::vector< int > > Plateau3D::lignes_possibles () const
```

Retourne l'ensemble des lignes gagnantes possibles. *.

Renvoie

- Liste des lignes du plateau.

5.9.3.11 operator=()

```
Plateau3D & Plateau3D::operator= (
    const Plateau3D & other)
```

Opérateur d'affectation. *.

*

Paramètres

<code>other</code>	Plateau à copier. *
--------------------	---------------------

Renvoie

Référence vers le plateau courant.

Voici le graphe d'appel pour cette fonction :

**5.9.3.12 retirer_pion()**

```
int Plateau3D::retirer_pion (
    int longueur,
    int largeur)
```

Retire le dernier pion d'une colonne. *.

*

Paramètres

<i>longueur</i>	Coordonnée en longueur. *
<i>largeur</i>	Coordonnée en largeur. *

Renvoie

Hauteur du pion retiré.

Voici le graphe d'appel pour cette fonction :



5.9.3.13 verifier_victoire()

```

bool Plateau3D::verifier_victoire (
    int x,
    int y,
    int z,
    Pion p) const
  
```

Vérifie si un joueur a gagné. *.

*

Paramètres

<i>x</i>	Coordonnée en longueur. *
<i>y</i>	Coordonnée en largeur. *
<i>z</i>	Coordonnée en hauteur. *
<i>p</i>	Pion du joueur. *

Renvoie

true si une victoire est détectée.

La documentation de cette classe a été générée à partir des fichiers suivants :

- src/Plateau3D.h
- src/Plateau3D.cpp

5.10 Référence de la classe position

Fonctions membres publiques

- **position** (char h='A', int v=0, int l=0)
- **position** (const [position](#) &o)
- [position](#) & **operator=** (const [position](#) &o)
- char **geth** ()
- int **getv** ()
- int **getl** ()
- char **getalpha** (int)
- void **seth** (char A)
- void **setv** (int v)
- void **affiche** ()
- int **positionNumMPI** ()

Fonctions membres publiques statiques

- static [position](#) **NumPosition** (int)

Attributs protégés

- char **h**
- int **v**
- int **l**

Attributs protégés statiques

- static int **n** =4
- static int **m** =5
- static vector< char > **alpha** ={'A','B','C','D'}

Amis

- ostream & **operator**<< (ostream &sortie, [position](#) p)

La documentation de cette classe a été générée à partir des fichiers suivants :

- src/position.h
- src/position.cpp

5.11 Référence de la structure Vec3

Représente une direction en 3 dimensions.

```
#include <Plateau3D.h>
```

Attributs publics

- int **dx**
Direction selon l'axe x.
- int **dy**
Direction selon l'axe y.
- int **dz**
Direction selon l'axe z.

5.11.1 Description détaillée

Représente une direction en 3 dimensions.

La documentation de cette structure a été générée à partir du fichier suivant :

- src/Plateau3D.h

Chapitre 6

Documentation des fichiers

6.1 arbitre.h

```
00001 /*
00002  * arbitre.h
00003  *
00004  * Created on: 30 avr. 2026
00005  * Author: dtromeur
00006  */
00007
00008 #ifndef ARBITRE_H_
00009 #define ARBITRE_H_
00010 #include <list>
00011 #include "position.h"
00012 #include "mpi.h"
00013 class arbitre {
00014     list<position> partie;
00015 public:
00016     arbitre();
00017     virtual ~arbitre();
00018     arbitre(const arbitre &other);
00019     arbitre& operator=(const arbitre &other);
00020
00021     position receiveCoup( int rank);
00027 void sendCoup(position, int rank);
00033 };
00034
00035 #endif /* ARBITRE_H_ */
```

6.2 IA_Aleatoire.h

```
00001 #ifndef IA_ALEATOIRE_H
00002 #define IA_ALEATOIRE_H
00003
00004 #pragma once
00005 #include "JoueurIA.h"
00006 #include "Pion.hpp"
00007 #include <string>
00008 #include <random>
00009
00010 using namespace std;
00011
00019 class IA_Aleatoire : public JoueurIA
00020 {
00021 public:
00026     IA_Aleatoire(Pion couleur);
00027
00033     IA_Aleatoire(string nom, Pion couleur);
00034
00039     IA_Aleatoire( const IA_Aleatoire & other);
00040
00044     ~IA_Aleatoire();
00050     IA_Aleatoire & operator=( const IA_Aleatoire & other);
00051
00057     std::pair<int, int> choisirCoup(const Plateau3D& plateau);
00058
00059 protected:
00060     mt19937 moteur;
00061
00062 };
00063
```

```
00064 #endif
```

6.3 IA_MinMax.h

```
00001 #ifndef IA_MINMAX_H
00002 #define IA_MINMAX_H
00003
00004 #pragma once
00005 #include <string>
00006 #include "Pion.hpp"
00007 #include "JoueurIA.h"
00008
00017 class IA_MinMax : public JoueurIA
00018 {
00019 public:
00026     IA_MinMax(std::string nom, Pion couleur, int ProfondeurEntree);
00027
00032     IA_MinMax( Pion couleur);
00033
00038     IA_MinMax( const IA_MinMax & other);
00039
00043     ~IA_MinMax();
00044
00050     IA_MinMax & operator=( const IA_MinMax & other);
00051
00057     std::pair<int, int> choisirCoup(const Plateau3D& plateau);
00058
00064     int evaluerPlateau(const Plateau3D& plateau);
00065
00072     int evaluer(int nbrPions, int Pions_adverse) const;
00073
00083     int min_max(Plateau3D& plateau, int profondeur, int alpha, int beta, bool estMax);
00084
00085 protected:
00086     static constexpr int ordreX[] = {2, 1, 3, 0, 4}; // je priorise les lignes aux centre et je met dans
une liste à la main
00087     static constexpr int ordreY[] = {1, 2, 0, 3}; // je priorise les colonnes au centre et je met dans
une liste à la main
00088     int profondeur;
00089
00090
00091 };
00092
00093 #endif
```

6.4 Joueur.h

```
00001 #ifndef JOUEUR_H
00002 #define JOUEUR_H
00003
00004 #pragma once
00005
00006 #include "Pion.hpp"
00007 #include "Plateau3D.h"
00008 #include <string>
00009
00018
00019 class Joueur
00020 {
00021 public:
00022     Joueur(std::string nom, Pion couleur);
00023     Joueur( Pion couleur);
00024     Joueur( const Joueur & other);
00025     Joueur & operator=( const Joueur & other);
00026
00027     virtual ~Joueur();
00028
00034     virtual std::pair<int, int> choisirCoup(const Plateau3D& plateau) =0 ;
00039     std::string getNom() const;
00040
00045     Pion getCouleur() const ;
00046
00047 protected:
00048     std::string nom;
00049     Pion couleur;
00050 };
00051
00052 #endif
```

6.5 JoueurHumain.h

```

00001 #ifndef JOUEURHUMAIN_H
00002 #define JOUEURHUMAIN_H
00003
00004 #pragma once
00005
00006 #include "Joueur.h"
00007 #include "Pion.hpp"
00008 #include <string>
00009 using namespace std;
00010
00017 class JoueurHumain :public Joueur
00018 {
00019 public:
00025     JoueurHumain(string nom, Pion couleur);
00026
00031     JoueurHumain( const JoueurHumain & other);
00032
00036     ~JoueurHumain();
00037
00043     JoueurHumain & operator=( const JoueurHumain & other);
00044
00050     std::pair<int, int> choisirCoup(const Plateau3D& plateau);
00051
00052 protected:
00053
00054 };
00055
00056 #endif

```

6.6 JoueurIA.h

```

00001 #ifndef JOUEURIA_H
00002 #define JOUEURIA_H
00003
00004 #pragma once
00005
00006 #include "Joueur.h"
00007 #include "Plateau3D.h"
00008 #include "Pion.hpp"
00009 #include <string>
00010
00011 using namespace std;
00012
00020 class JoueurIA : public Joueur
00021 {
00022 {
00023 public:
00028     JoueurIA(Pion couleur );
00029
00035     JoueurIA( string nom , Pion couleur);
00036
00041     JoueurIA( const JoueurIA & other);
00042
00046     ~JoueurIA();
00047
00053     JoueurIA & operator=( const JoueurIA & other);
00054
00055 protected:
00056
00057 };
00058
00059 #endif

```

6.7 JoueurMPI.h

```

00001 /*
00002  * joueur.h
00003  *
00004  * Created on: 29 avr. 2026
00005  * Author: dtromeur
00006  */
00007
00008 #ifndef JOUEURMPI_H_
00009 #define JOUEURMPI_H_
00010 #include <iostream>
00011 #include <list>
00012 #include "position.h"
00013 #include "mpi.h"

```

```

00014 using namespace std;
00015 class JoueurMPI {
00023     int me;
00024     int color;
00025 public:
00026     JoueurMPI(int color);
00027     virtual ~JoueurMPI();
00028     JoueurMPI(const Joueur &other);
00029     JoueurMPI& operator=(const Joueur &other);
00030
00031
00032     position receiveCoup( int =2);
00037     void sendCoup(position, int =2);
00042 };
00043
00044 #endif /* JOUEURMPI_H_ */

```

6.8 Référence du fichier src/main.cpp

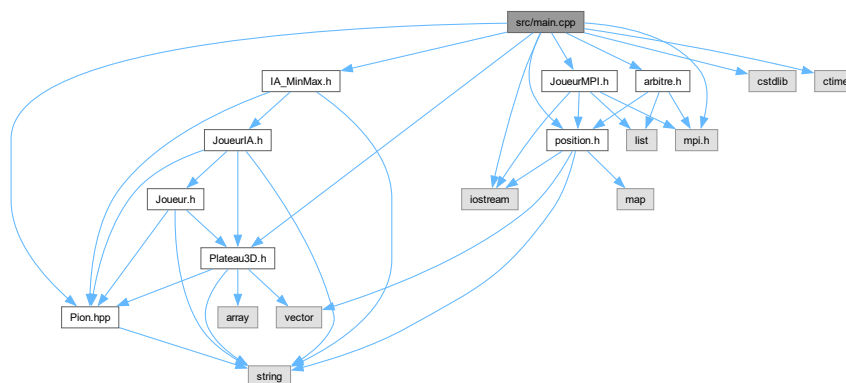
Programme principal utilisant MPI.

```

#include <iostream>
#include <cstdlib>
#include <ctime>
#include "mpi.h"
#include "Pion.hpp"
#include "Plateau3D.h"
#include "IA_MinMax.h"
#include "arbitre.h"
#include "JoueurMPI.h"
#include "position.h"

```

Graphe des dépendances par inclusion de main.cpp:



Fonctions

- `position versPosition` (int x, int y)
- `std::pair< int, int > versCoordonnees` (`position` p)
- `position signalFin` ()
- `bool estSignalFin` (`position` p)
- `int main` (int argc, char *argv[])

6.8.1 Description détaillée

Programme principal utilisant MPI.

Ce fichier lance une partie entre deux IA MinMax qui communiquent avec un arbitre grâce à MPI.

6.9 Partie.h

```

00001 #ifndef PARTIE_H
00002 #define PARTIE_H
00003
00004 #pragma once
00005 #include "Joueur.h"
00006 #include "Plateau3D.h"
00007
00018 class Partie
00019 {
00020 public:
00024     Partie();
00025
00031     Partie(Joueur* J1, Joueur* J2);
00032
00037     Partie(const Partie & other);
00038
00042     ~Partie();
00043
00049     Partie & operator=(const Partie & other);
00050
00060     bool victoire_tour(Joueur *j);
00061
00068     void lancer();
00069
00075     Joueur* lancer_silencieux(bool j1commence);
00076
00081     const Plateau3D& getPlateau() const{
00082         return this->plateau;
00083     }
00084
00085 protected:
00086     Joueur* J1;
00087     Joueur* J2;
00088     Plateau3D plateau;
00089     int tours_joues = 0;
00090 };
00091
00092 #endif

```

6.10 Pion.hpp

```

00001 #ifndef PION_HPP
00002 #define PION_HPP
00003
00004 #include <string>
00005
00010
00011 enum class Pion {
00012     Vide,
00013     Blanc,
00014     Noir
00015 };
00016
00022
00023 inline std::string toString(Pion p){
00024     switch (p){
00025         case Pion::Blanc: return "O";
00026         case Pion::Noir : return "X";
00027         default : return ".";
00028     }
00029 }
00030
00036
00037 inline Pion operator!(Pion p){
00038     switch (p)
00039     {
00040         case Pion::Blanc: return Pion::Noir;
00041         case Pion::Noir:  return Pion::Blanc;
00042         default:
00043             return Pion::Vide;
00044     }
00045 }
00046
00047 #endif

```

6.11 Plateau3D.h

```

00001 #ifndef PLATEAU3D_H

```

```

00002 #define PLATEAU3D_H
00003
00004 #include <array>
00005 #include <string>
00006 #include <vector>
00007 #include "Pion.hpp"
00008
00009
00010
00011
00012
00013
00014 struct Vec3 {
00015     int dx;
00016     int dy;
00017     int dz;
00018 };
00019
00020
00021
00022
00023
00024
00025
00026
00027
00028 class Plateau3D
00029 {
00030
00031 public:
00032     Plateau3D();
00033     ~Plateau3D();
00034     Plateau3D(const Plateau3D &other);
00035     Plateau3D& operator=(const Plateau3D &other);
00036
00037     bool est_coup_valide(int x, int y) const;
00038
00039     int ajouter_pion(int longueur, int largeur, Pion p);
00040     int retirer_pion(int longueur, int largeur);
00041
00042     void affiche();
00043
00044     bool verifier_victoire(int x, int y, int z, Pion p) const;
00045
00046     int get_largeur() const;
00047     int get_longueur() const;
00048     int get_total_emplacements() const;
00049
00050     std::vector<std::vector<int>> lignes_possibles() const;
00051     Pion get_pion(int hauteur, int largeur, int longueur) const ;
00052     Pion get_pion(int index) const;
00053
00054     bool est_termine();
00055
00056 private:
00057     static constexpr int hauteur = 4;
00058     static constexpr int largeur = 4;
00059     static constexpr int longueur = 5;
00060
00061     std::array<Pion, hauteur * largeur * longueur> grille;
00062     bool plateau_termine;
00063     int nbr_pions_places;
00064
00065     static constexpr Vec3 directions[] = {
00066         {1,0,0}, {0,1,0}, {0,0,1}, // Axes
00067         {1,1,0}, {1,-1,0}, {1,0,1}, {1,0,-1}, {0,1,1}, {0,1,-1}, // Diagonales faces
00068         {1,1,1}, {1,1,-1}, {1,-1,1}, {1,-1,-1} // Diagonales spatiales
00069     };
00070
00071     std::vector<std::vector<int>> LIGNES_POSSIBLES;
00072     void initialiser_lignes();
00073
00074     int index(int longueur, int largeur, int hauteur) const;
00075
00076     void def_pion(int hauteur, int largeur, int longueur, Pion p);
00077
00078     int compter_pions_direction(int x, int y, int z, int dx, int dy, int dz, Pion p) const;
00079 };
00080
00081 #endif

```

6.12 position.h

```

00001 /*
00002  * position.h
00003  *
00004  * Created on: 6 avr. 2026
00005  * Author: dtromeur
00006  */
00007
00008 #ifndef POSITION_H_
00009 #define POSITION_H_
00010 #include <vector>

```

```

00011 #include <map>
00012 #include <string>
00013 #include <iostream>
00014 using namespace std;
00015 class position {
00016 protected:
00017     static int n;
00018     static int m;
00019     static vector<char> alpha;
00020     char h;
00021     int v;
00022     int l;
00023 public:
00024     position(char h='A', int v=0, int l=0);
00025     position(const position &o);
00026     position& operator=(const position &o);
00027     virtual ~position();
00028
00029     static position NumPosition(int);
00035     char geth();
00036     int getv();
00037     int getl();
00038     char getalpha(int);
00039
00040     void seth(char A);
00041     void setv(int v);
00042
00043     void affiche();
00044     int positionNumMPI();
00050     friend ostream & operator<<(ostream & , position p);
00051
00052 };
00053
00054 #endif /* POSITION_H_ */

```

6.13 Référence du fichier src/test.cpp

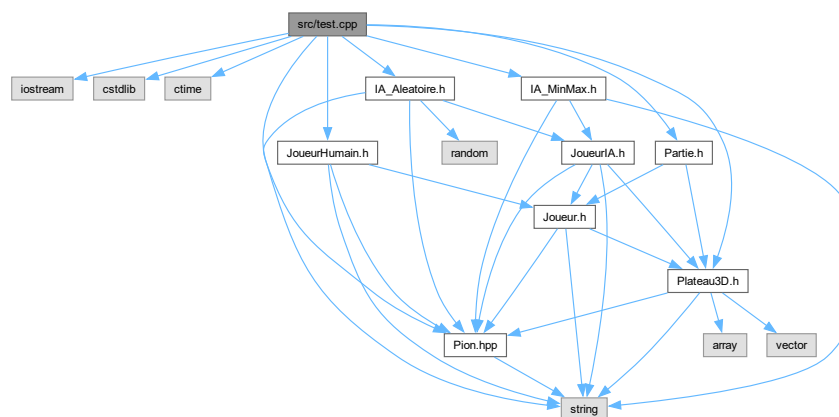
Fichier de test pour comparer les IA.

```

#include <iostream>
#include <cstdlib>
#include <ctime>
#include "Pion.hpp"
#include "Plateau3D.h"
#include "JoueurHumain.h"
#include "IA_Aleatoire.h"
#include "Partie.h"
#include "IA_MinMax.h"

```

Graphe des dépendances par inclusion de test.cpp:



Fonctions

- void **visualiserLignes** (const [Plateau3D](#) &plateau)
- int **main** ()

6.13.1 Description détaillée

Fichier de test pour comparer les IA.

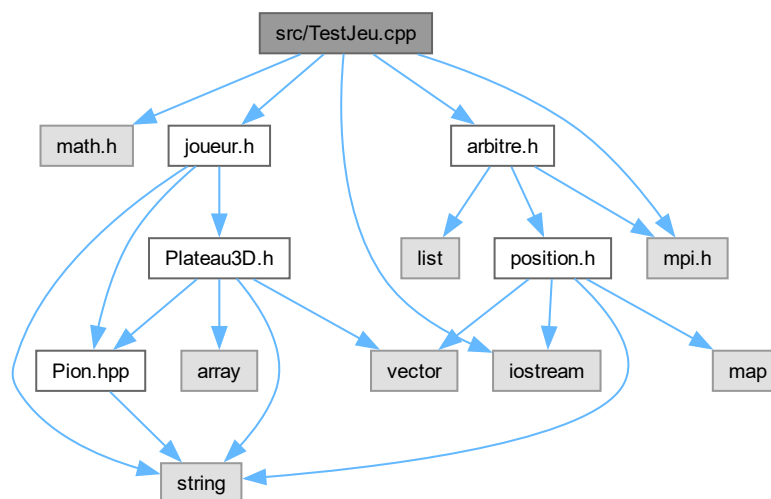
Ce fichier permet de lancer plusieurs parties automatiquement afin de comparer les performances des différentes IA.

6.14 Référence du fichier src/TestJeu.cpp

Ancien fichier de test pour les communications MPI.

```
#include <math.h>
#include <iostream>
#include "mpi.h"
#include "arbitre.h"
#include "joueur.h"
```

Graphe des dépendances par inclusion de TestJeu.cpp:



Fonctions

- int **main** (int argc, char *argv[])

6.14.1 Description détaillée

Ancien fichier de test pour les communications MPI.